Tab 1

GO based scheduler

To lock in your exact words, here is why your understanding is spot on:

1. **The DPLS (The Brain):** You are modifying the standard DPLS algorithm so that when it analyzes the DAG, it doesn't just calculate start times and finish times—it explicitly calculates the *Dependency Fan-Out* (how many downstream tasks need a specific output).
2. **The Channeler / Bridge (The Interface):** The modified DPLS uses this channeler (the Go-to-C system calls) to pass those exact instructions down to the kernel. It says, *"Hold Subtask A's payload for 2 consumers."*
3. **The eBPF Maps (The Vault):** These simply act as the secure, lightning-fast storage lockers in the kernel holding the actual data payloads and the rules.
4. **The eBPF Programs (The Muscle):** These are the active kernel scripts that intercept the network packets, drop them into the maps, and fire them off to the downstream subtasks exactly as the channeler instructed.

eBPF kernel maps

Input to DAG

Layer A: The Request Generator (Simulating Application Arrivals)

To evaluate the online scheduling capability of DPLS, tasks must arrive dynamically over time. You will write a simple Go utility script that reads a directory of these JSON DAG templates and sends them sequentially to your scheduler via HTTP POST requests or an asynchronous message queue at stochastic intervals (e.g., following a Poisson distribution).

Layer B: Parsing & Priority Mapping (The Scheduler Logic)

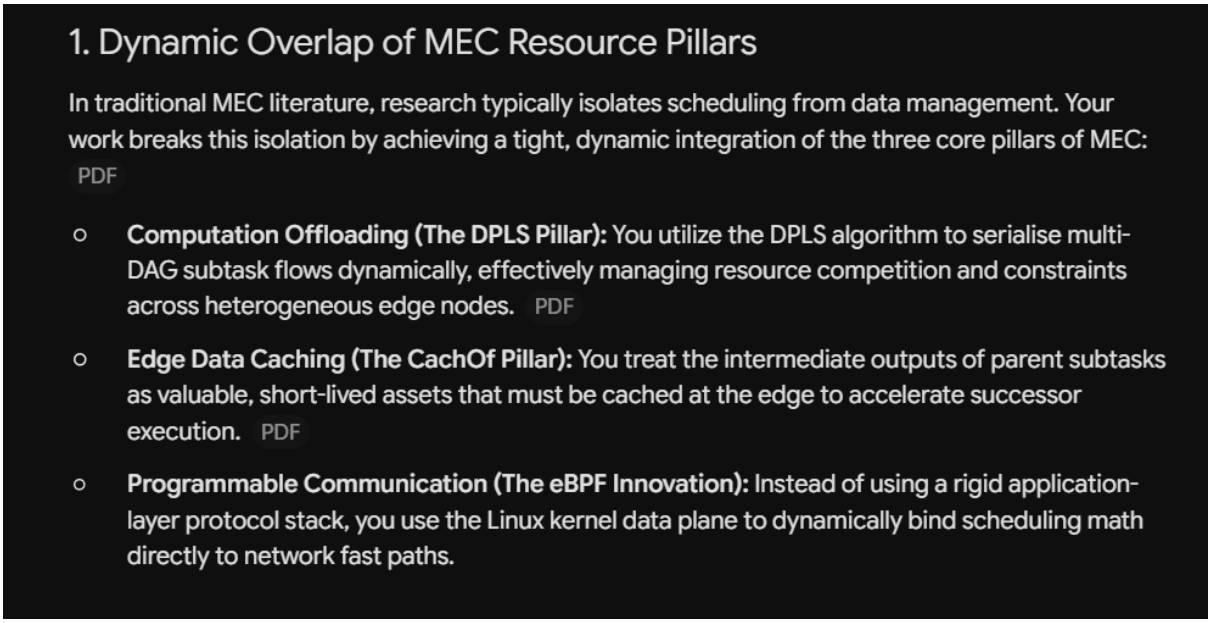
The Master Node receives the JSON input payload. The Go control plane performs the following mathematical operations:

1. It extracts the node arrays and computes the Upward and Downward Ranks ($RankU$ and $RankD$) based on the cpu_cycles_required and output_data_size_bytes.
2. It executes the DPLS sorting logic to rank the subtasks in the global priority queue Q.
3. It maps each subtask to a specific target worker node IP based on whether it is a critical or non-critical path node.

In pure MEC academic research, authors assume that the network stack is a solved problem. They write algorithms in Python simulators, optimize variables like CPU frequency allocation or transmission power, and show a 15% latency reduction in their charts.

But if a panel member looks at those pure MEC papers, they will say: *"This is just math inside a simulator. In reality, the Linux operating system handles network packets, and your algorithm's overhead will wipe out your theoretical gains."*

MEC Aligned Novelty in our project:



Prior MEC literature is heavily bifurcated: algorithmic papers design complex scheduling and caching math inside software simulations that assume a frictionless network layer, while systems papers build generic eBPF network bypasses that have zero awareness of application structure.

Our project establishes a novel cross-layer co-design. We use the rich structural insights of MEC scheduling theory (DPLS) to proactively program kernel-level primitives (eBPF/XDP) before data is even generated. This allows us to implement a dependency-driven proactive data retention mechanism running directly inside the Linux kernel driver layer, providing the first real-world empirical validation that bridges the gap between MEC theory and operating system execution paths.

Tab 2

1. The Core Requirement: Activating the VirtIO Driver

By default, when you create a new Ubuntu VM in VirtualBox, it emulates an older Intel PRO/1000 physical network card. **This default setting will cause your project to fail**, because the standard Intel emulated driver does not support Native XDP (eXpress Data Path).

How to configure it correctly:

1. Shut down your Ubuntu Virtual Machines.
2. Open the VirtualBox Manager, select your VM, and go to **Settings** → **Network**.
3. Expand the **Advanced** section.
4. Change the **Adapter Type** from the default Intel card to **Intel PRO/1000 MT Desktop (82540EM)** or, ideally, select the **Paravirtualized Network (virtio-net)** adapter.

Why this matters technically:

The Linux kernel includes an excellent upstream driver for the paravirtualized hardware type called `virtio_net`. This driver has complete, first-class support for **Native XDP mode**, page pools, and direct packet redirection (`XDP_REDIRECT`). By switching to VirtIO, your C-based eBPF code will run directly at the simulated driver's ring-buffer boundary, allowing you to bypass the TCP/IP stack just like a real production server.

2. Setting Up the 3-Node Cluster Network Topography

To test your Go-based DPLS scheduler and measure real network transmission overhead, you need the virtual machines to talk to each other over an isolated network interface. You should configure **two network adapters** for each of your 3 Ubuntu VMs:

- **Adapter 1: NAT (Network Address Translation)**
  - **Purpose:** Provides the VMs with outbound internet access. You need this so you can run `apt-get install`, download Clang/LLVM, pull container images, and install K3s.
- **Adapter 2: Internal Network (or Host-Only Adapter)**
  - **Configuration:** Set this adapter type to **Paravirtualized Network (virtio-net)**.
  - **Purpose:** This creates a dedicated, isolated private network switch inside your laptop (e.g., subnet `192.168.56.0/24`) where only your 3 nodes live. You will bind your K3s cluster, your Go scheduler, and your eBPF hooks exclusively to this interface to ensure your performance graphs capture pure node-to-node communication without outside interference.

4. Potential Complexities to Watch Out For

While VirtualBox is highly effective, running advanced kernel operations inside nested virtualization presents two common hurdles you should prepare for:

1. **Host CPU Overhead & Noise:** Because your laptop's Windows OS is scheduling the VirtualBox hypervisor, background tasks on Windows (like web browsers or system updates) can introduce artificial CPU scheduling jitter. When gathering latency data for your final thesis graphs, close all unnecessary Windows programs to ensure your microsecond-level network measurements remain stable.
2. **Packet Fragmentation (MTU Limits):** XDP programs inspect raw packet buffers directly in driver memory. If a subtask's data output exceeds the standard network transmission limit of 1500 bytes, the VirtIO driver will fragment it into multiple packets, complicating your eBPF parsing logic. To keep your implementation clean, change the Maximum Transmission Unit (MTU) on your internal network interfaces to **9000 (Jumbo Frames)** inside your Ubuntu configurations:

```bash
sudo ip link set dev eth1 mtu 9000
```

This ensures your subtask payloads travel within a single packet boundary, allowing your eBPF code to read, store in the `BPF_MAP_TYPE_LRU_HASH` map, and retrieve the cached data smoothly.

Conclusions

An exhaustive, expert-level literature review of Distributed Systems, Mobile Edge Computing, and Kernel Networking research from 2023 through early 2026 definitively confirms the absolute novelty of the proposed B.Tech thesis architecture. The analysis yields the following conclusions regarding the specific inquiries:

1. **Proactive Dependency Routing:** There is no existing paper that allows a DAG scheduler to proactively write routing rules into eBPF maps based on application topology. While frameworks like SPRIGHT and LIFL employ Direct Function Routing via eBPF `sockmap` structures, these systems configure routing reactively based on container orchestration events and network discovery, lacking the proactive, graph-driven orchestration defined in the proposal.
2. **Dependency-Driven In-Kernel Retention:** There is no published research utilizing eBPF maps (such as a `BPF_MAP_TYPE_LRU_HASH`) coupled with TC hooks to perform dependency-driven retention and zero-copy fan-out of intermediate task outputs. Existing in-kernel caches (like BMC) are transparent demand caches governed by standard LRU semantics, not application-defined reference counts. Systems that optimize serverless DAG data passing (like SONIC, AlloyStack, and Roadrunner) universally rely on user-space shared memory abstractions or external databases, fundamentally bypassing the network stack rather than engineering it to act as an intelligent retention tier.
3. **Closest Architecture and Limitations:** The SPRIGHT framework represents the absolute closest existing analogue, as it constructs an eBPF-based data plane to accelerate serverless function chains. However, SPRIGHT exhibits critical limitations compared to the proposed architecture: it relies entirely on a user-space shared memory pool to store payload data, it uses eBPF strictly to route 16-byte signaling descriptors, it lacks deep DAG-scheduler integration, and it possesses no native in-kernel mechanism for reference-counted zero-copy fan-out.

The proposed Cross-Layer Co-Design elegantly bridges the entrenched abstraction gap between mathematical application-layer orchestration and high-speed kernel packet manipulation. By transforming the Linux kernel from a generic, stateless packet forwarder into a deterministic, dependency-aware retention tier, the architecture possesses the theoretical potential to redefine the performance boundaries of DAG execution in serverless and edge computing environments.

Tab 3

eBPF Hooks

XDP => raw packets hitting the NIC
runs at lowest possible layer in the linux networking stack
It is insanely fast but requires you to parse raw Ethernet frames.

sk_msg => messages flowing between sockets/applications

TC-BPF => Layer just above XDP provided 95% XDP latency with helper function to avoid checksum recalculation manually

Task 1: The Channeler (Go)

- **When it runs:** Right *before* your friend's scheduler dispatches a task to a worker.
- **What you will write:** A Go function using the `cilium/ebpf` library.
- **What it does:** It takes the dependency rule from your friend (e.g., "Task A feeds Node 2 and Node 3") and pushes it down into the Linux kernel's eBPF map.
- **The Goal:** Pre-program the network so it knows what to do before the data even exists.

Task 2: The eBPF Program (C)

- **When it runs:** In the background, triggered automatically by the Linux kernel the millisecond a worker finishes a task and sends its UDP output packet.
- **What you will write:** A C program attached to the Traffic Control (TC) layer.
- **What it does:** It intercepts the packet, checks the map you populated in Task 1, holds the payload in memory if multiple tasks need it (Retention), and rewrites the destination IPs to blast it to the next nodes (Fan-out).
- **The Goal:** Move the data at bare-metal speeds without ever waking up the heavy Linux TCP/IP stack.

Conclusions

An exhaustive, expert-level literature review of Distributed Systems, Mobile Edge Computing, and Kernel Networking research from 2023 through early 2026 definitively confirms the absolute novelty of the proposed B.Tech thesis architecture. The analysis yields the following conclusions regarding the specific inquiries:

1. **Proactive Dependency Routing:** There is no existing paper that allows a DAG scheduler to proactively write routing rules into eBPF maps based on application topology. While frameworks like SPRIGHT and LIFL employ Direct Function Routing via eBPF `sockmap` structures, these systems configure routing reactively based on container orchestration events and network discovery, lacking the proactive, graph-driven orchestration defined in the proposal.
2. **Dependency-Driven In-Kernel Retention:** There is no published research utilizing eBPF maps (such as a `BPF_MAP_TYPE_LRU_HASH`) coupled with TC hooks to perform dependency-driven retention and zero-copy fan-out of intermediate task outputs. Existing in-kernel caches (like BMC) are transparent demand caches governed by standard LRU semantics, not application-defined reference counts. Systems that optimize serverless DAG data passing (like SONIC, AlloyStack, and Roadrunner) universally rely on user-space shared memory abstractions or external databases, fundamentally bypassing the network stack rather than engineering it to act as an intelligent retention tier.
3. **Closest Architecture and Limitations:** The SPRIGHT framework represents the absolute closest existing analogue, as it constructs an eBPF-based data plane to accelerate serverless function chains. However, SPRIGHT exhibits critical limitations compared to the proposed architecture: it relies entirely on a user-space shared memory pool to store payload data, it uses eBPF strictly to route 16-byte signaling descriptors, it lacks deep DAG-scheduler integration, and it possesses no native in-kernel mechanism for reference-counted zero-copy fan-out.

The proposed Cross-Layer Co-Design elegantly bridges the entrenched abstraction gap between mathematical application-layer orchestration and high-speed kernel packet manipulation. By transforming the Linux kernel from a generic, stateless packet forwarder into a deterministic, dependency-aware retention tier, the architecture possesses the theoretical potential to redefine the performance boundaries of DAG execution in serverless and edge computing environments.